Classification of Instruments Using Mel-Spectrograms and Convolutional Neural Networks

Srinilai Kankipati, Joshua Yu

December 6, 2024

Abstract

Classification of audio is an artificial intelligence task that lies at the heart of many useful technologies such as wake-word detection with voice assistants (Siri, Alexa, Google), music detection, adaptive music mixing, scene analysis for autonomous driving, and more. The applications are immense. One may be surprised to learn that audio classification is typically a computer vision problem! Audio can be represented in many ways. Crucially, it can be represented visually as spectrograms. This allows for the classification of audio samples with convolutional neural networks. In this paper, the classification of instruments for use in adaptive mixing will be explored. Several classes of instruments and performance sounds will be classified with a variety of pretrained convolutional neural networks and the results will be compared and analyzed.

1 Introduction

Music production and engineering involves manually mixing separate instruments and sounds to create a cohesive and pleasing sound. Mixing is an involved process that involves balancing of levels, control of dynamics and frequency content, and application of creative effects to enhance the interest and musicality of the sound. Music mixing is a highly-dimensional process, and even a professional engineer may require several hours or days to mix a single song. With the boom in artificial intelligence over the last decade, much research has been done into the automatic mixing of music to expedite the music creation process and provide the ability to obtain a professional-sounding mix to the average musician that does not have access to a professional engineer.

One crucial, if not obvious element to mixing is the classification of each individual source sound. The classification of source sounds allows for the appropriate placement of each element within the music mix. For example, a soft background vocal should not overpower a lead singer nor should a screaming solo guitar be buried behind the rhythm piano. There are two main dimensions of classification. One is the actual classification of the source sound. For instance, one may identify a singer, a guitar, a drum set, or a bass. It is also crucial to classify the role of a given sound; For instance, one must classify whether a vocalist is a lead singer or a background singer. The classification of these two dimensions allows a mix engineer to make informed decisions about how to balance volume and clarity, among other things.

The classification of sounds is typically approached as a visual task. Audio can be represented visually as spectrograms, which allows for the use of convolutional neural networks. Convolutional neural networks are well suited to the task of audio classification due to their ability to learn and and recognize patterns in spectrograms. By using this technology, the authors aim to enable the automatic classification of instruments and sounds for use in automatic mixing.

As mentioned above, the implications of such a system are significant. Not only does it expedite the workflow of professional engineers, but it also enables amateur musicians to produce professional sounding music without the cumbersome expense and time commitment of a professional engineer. The goal of audio classification and automatic mixing is to provide high-quality musical tools for everyone.

2 Background: Audio Representation

Before the actual classification process with convolutional neural networks is performed, there are several important points to be considered in the process of transforming audio into a visual representation.

Audio is typically represented as a spectrogram. A spectrogram is a visualization of the frequency content of a piece of audio through time. The x-axis represents time and the y-axis represents frequency. The intensity of audio at any given frequency is shown with a heatmap style representation.

One notable property of audio is that the frequency scale is logarithmic and does not scale well with human perception of pitch. With the frequency scale, a doubling of frequency will always result in a linear increase of one octave. Thus, a jump from 1000 Hz to 2000 Hz produces the same perceptual pitch difference as going from 2000 Hz to 4000 Hz. This produces rather lop-sided spectrograms which produce a heavy concentration of frequencies towards the bottom of the graph and do not match the typical linear perception of sound. One way to fix this problem is to represent frequencies on the Mel scale instead of the typical frequency scale (which uses Hertz). The Mel scale differs from the normal frequency scale in that its values scale linearly according to human perception of pitch. This produces spectrograms which are more evenly spread along the y-axis and provide more robust features for convolutional neural networks. These spectrograms are referred to as Mel-spectrograms. An example Mel-spectrogram of an acoustic guitar is shown below.

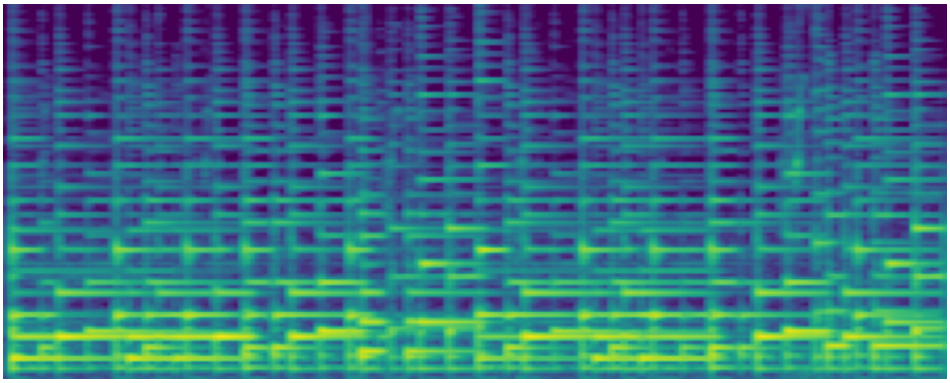


Figure 1: Mel-Spectrogram of an acoustic guitar

It is quite interesting to note that even to the untrained eye, it is possible to differentiate between different types of sounds. A comparison of several different instruments is shown below.

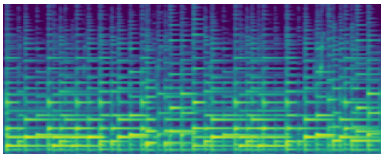


Figure 2: Acoustic Guitar

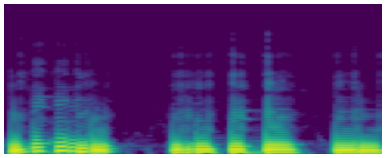


Figure 3: Bass

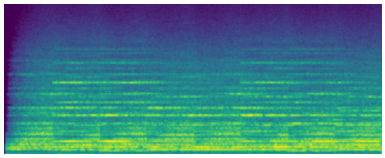


Figure 4: Background Pad

Three examples are shown, one of the acoustic guitar from above, one of a bass guitar, and one of a synthesized background pad. One can easily distinguish the bass by its lack of high-frequency content. The smooth synthesizer pad can be distinguished by its gradual changes over time as well as its filled-out frequency content. The acoustic guitar can be distinguished by its frequency content and transient nature; one can literally see the strums as they occur. Given that such features are easily distinguished by the naked eye, it follows that a convolutional neural network can reasonably pick up on such features and use them to classify audio sources. Mel-spectrograms are a robust audio representation method which will be useful in this experiment.

3 Method

For the experiment, three different pre-trained models will be used. Transfer learning will be performed on all three. The best-performing model will be fine-tuned and further analysis will be performed. The three models selected are ResNet18, EfficientNetB0, and VGG16. The three models are relatively compact and all accept the same input image size of 224x224.

ResNet18 is a convolutional neural network model introduced in 2015 by He et al. [HZRS15] As suggested by the name, the model architecture features 18 network layers and possesses approximately 11.5 million trainable parameters. [Anw19] It is unique in its use of residual blocks which skip layer connections to improve efficiency in training. This allows for the use of incredibly deep convolutional neural networks. At the time, it was a breakthrough in model accuracy. The model architecture is shown below.

layer name	output size	18-layer
conv1	112×112	
conv2_x	56×56	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 2$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 2$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 2$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 2$
	1×1	
FLOPs		1.8×10^9

Figure 5: The architecture of ResNet18, from [HZRS15].

The original ResNet18 model's final layer is a 1000 output fully-connected layer to accommodate all 1000 classes that the network can classify. In order to accommodate the requirements of this experiment, the fully connected layer is replaced with an intermediate fully-connected layer with 256 outputs as well as a final fully-connected layer to output the classification results for the five classes.

EfficientNet-B0 is one of a set of convolutional neural networks introduced in 2019 by Tan and Le. [TL19] EfficientNet as a whole is a novel model architecture which provides a new scaling method to uniformly scale all the dimensions of a neural network (depth, width, resolution). The result is a highly-accurate image classifier that outperforms networks like ResNet and VGG while being smaller and faster.

EfficientNet-B0 specifically is the smallest model out of the EfficientNet family of models. It is a convolutional neural network with 18 network layers and approximately 5.3 million trainable parameters. [TL19] The model architecture is outlined below.

Stage	Operator	Resolution	#Channels	#Layers
1	Conv3x3	224 × 224	32	1
2	MBConv1, k3x3	112 × 112	16	1
3	MBConv6, k3x3	112 × 112	24	2
4	MBConv6, k5x5	56 × 56	40	2
5	MBConv6, k3x3	28 × 28	80	3
6	MBConv6, k5x5	14 × 14	112	3
7	MBConv6, k5x5	14 × 14	192	4
8	MBConv6, k3x3	7 × 7	320	1
9	Conv1x1 & Pooling & FC	7 × 7	1280	1

Figure 6: The architecture of EfficientNet-B0, from [Gee24a]

The original EfficientNet-B0 model's final layer is a 1280 output fully-connected layer to accommodate all the classes that the network can classify. In order to accommodate the requirements of this experiment, the fully connected layer is replaced with an intermediate fully-connected layer with 256 outputs as well as a final fully-connected layer to output the classification results for the five classes.

VGG16 is a convolutional neural network model introduced in 2014 by Simonyan and Zisserman. It is one of the earlier examples of deep convolutional networks. It features 16 layers with a notably

large 138 million parameters. [SZ15]. The performance and speed is slightly outdated compared to the newer models described above. The model architecture is shown below.

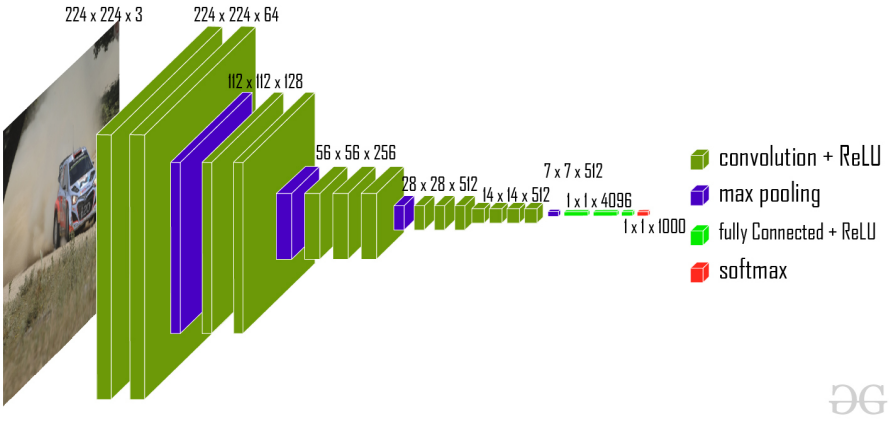


Figure 7: The architecture of VGG16, from [Gee24b]

The original VGG16 model's final layer is a 1000 output fully-connected layer to accommodate all the classes that the network can classify. In order to accommodate the requirements of this experiment, the fully connected layer is replaced with an intermediate fully-connected layer with 256 outputs as well as a final fully-connected layer to output the classification results for the five classes.

4 Experiments

4.1 Dataset

No existing dataset covers both dimensions of classification as described in the introduction. As such, the authors constructed a miniature dataset based on the audio samples offered by the music production software Logic Pro. The dataset has 700 pieces of data and covers five categories. the five categories are acoustic guitar, electric guitar, bass, lead synthesizer, pad synthesizer. These categories cover the two dimensions by representing source detection with different instruments (acoustic guitar, electric guitar, bass) as well as role detection (lead synthesizer, pad synthesizer). The dataset contains 701 pieces of spectrogram data. The data is split across the categories as shown in the following table.

Classes	Acoustic Guitar	Electric Guitar	Bass	Lead Synthesizer	Pad Synthesizer
Quantity	184	216	100	100	101

Table 1: Dataset Composition

The dataset was created by manually selecting audio samples from Logic Pro. To process the audio into the final Mel-spectrogram representation, all the audio was first converted into mono audio with a sampling rate of 16000 Hertz. Once the audio was standardized, a Python script was used to convert the audio into Mel-spectrograms. The Python libraries Librosa and Matplotlib were used to convert the audio and visualize the results, respectively. Some sample spectrograms from each class are shown below.

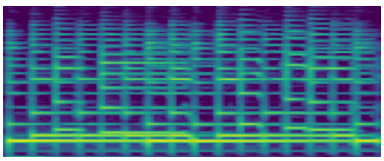


Figure 8: Acoustic Guitar

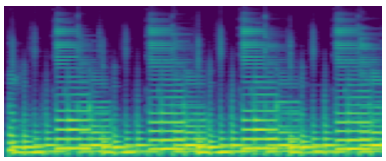


Figure 9: Electric Guitar

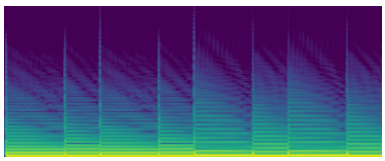


Figure 10: Bass

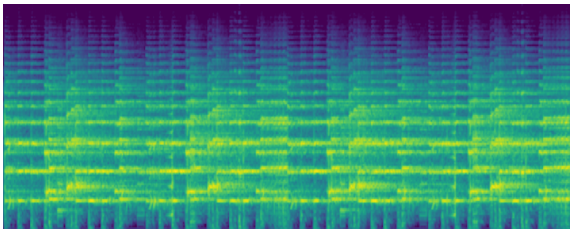


Figure 11: Lead Synthesizer

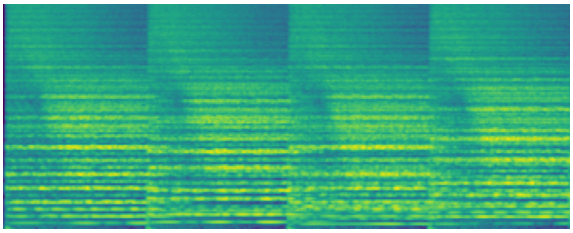


Figure 12: Pad Synthesizer

The data set was split up for training and testing purposes. The dataset was split into train and test categories with a 0.2 test split using the scikit-learn Python library. The resulting split is shown below.

Classes	Acoustic Guitar	Electric Guitar	Bass	Lead Synthesizer	Pad Synthesizer
Train	147	172	80	80	80
Test	37	44	20	20	21

Table 2: Dataset Split

The spectrogram output from Librosa and Matplotlib was 775x308, however, PyTorch transforms were used to transform the data into the appropriate size of 224x224 for input into the pre-trained models. Additionally, the data was normalized to align with the mean pixel values and image channel standard deviation values of the ImageNet dataset. This stabilizes training and ensures features are scaled similarly to the dataset that the pre-trained models were trained on.

4.2 Evaluation metrics

To rigorously assess the performance of our audio classification models, we employ a comprehensive set of evaluation metrics tailored to measure both the learning process and the generalization capabilities of the models. Given that our task involves classifying mel-spectrogram representations of different instruments, the chosen metrics provide valuable insights into the accuracy and reliability of the classification outcomes. The primary evaluation metrics utilized in this study are detailed below:

Firstly, we will evaluate training loss. Training loss quantifies the discrepancy between the model's predictions and the actual labels during the training phase. In our implementation, we use the Cross-Entropy Loss function, which is well-suited for multi-class classification problems. For each epoch, the loss is computed as the average cross-entropy loss over all batches in the training dataset:

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

where:

- N is the number of samples,
- C is the number of classes,
- $y_{i,c}$ is the ground truth label (one-hot encoded),
- $\hat{y}_{i,c}$ is the predicted label.

Monitoring the training loss helps in understanding how well the model is learning from the training data. A decreasing training loss over epochs typically indicates that the model is improving its fit to the training data. Conversely, stagnation or an increase in training loss may signal issues such as inadequate model capacity or learning rate problems.

We will also utilize test accuracy to evaluate our methods. Test accuracy measures the proportion of correctly classified samples in the test dataset, providing a straightforward indicator of the model's overall performance.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} * 100\%$$

In our implementation, for each batch in the test dataset, the model's predictions are compared against the true labels, and the number of correct predictions is accumulated to compute the overall accuracy. Test accuracy serves as a primary metric for evaluating the model's ability to generalize to unseen data. High test accuracy indicates that the model effectively captures the underlying patterns in the data, whereas low accuracy may suggest overfitting, underfitting, or challenges in distinguishing between classes.

While training loss and test accuracy provide essential insights, incorporating precision, recall, F1-score, and confusion matrices offers a more nuanced understanding of our model's performance. Precision is the ratio of true positive predictions to the total predicted positives for each class. Recall is the ratio of true positive predictions to the actual positives for each class. F1-score is the harmonic mean of precision and recall, providing a balance between the two.

For each class c:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad \text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad \text{F1-Score}_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

where:

- TP_c = True Positives,
- FP_c = False Positives,
- FN_c = False Negatives.

These metrics help in understanding the model's performance on individual classes, especially in cases where class imbalance exists or certain classes are more critical than others.

A confusion matrix is a tabular representation that illustrates the performance of the classification model by showing the actual versus predicted classifications for each class. The matrix is constructed with rows representing the true classes and columns representing the predicted classes. Each cell (i,j) indicates the number of samples from true class i that were predicted as class j. Analyzing the confusion matrix helps in identifying specific classes that the model may be confusing, thereby highlighting areas for potential improvement, such as data augmentation for underrepresented classes or refining model architecture.

Beyond quantitative metrics, assessing model robustness involves evaluating its performance under varying conditions, such as different noise levels in audio samples or variations in instrument timbres. To further validate our model's generalization capabilities, we will also be utilizing cross-validation. We'll also be using visual plots to help visualize the learning trajectory and to identify issues like overfitting or underfitting.

4.3 Results

The dataset used in this research consists of mel-spectrogram representations of various instrument classes. Each audio sample was preprocessed to generate spectrograms, which serve as the input to the models. The dataset was divided into training and test sets using an 80-20 split. The training set contained approximately N_{train} samples, while the test set included N_{test} samples. These spectrograms were standardized using the ImageNet normalization parameters to ensure compatibility with pretrained convolutional neural network architectures.

Three state-of-the-art pretrained CNN architectures—ResNet18, EfficientNet-B0, and VGG16—were employed for this task. Transfer learning was used by fine-tuning the final classification layers of each model to adapt them to the instrument classification problem.

ResNet18 achieved a final test accuracy of 88.03% after 10 epochs. While this is slightly lower than initially expected, ResNet18 still performed relatively well compared to the other models. The model showed steady improvement in accuracy over the epochs, indicating its strong capacity to learn discriminative features from the dataset. The relatively high performance can be attributed to the

effective residual connections in ResNet18, which help mitigate issues such as vanishing gradients, allowing the model to learn deeper features without significant performance degradation.

EfficientNet-B0 reached an accuracy of 86.62%, which is lower than that of ResNet18 but still a solid result. While EfficientNet-B0 demonstrated quick improvements in the early epochs, its performance plateaued earlier compared to ResNet18, suggesting that the model's architecture, which prioritizes computational efficiency, may have limited its ability to learn more complex patterns in the data.

VGG16, despite its deeper architecture, outperformed both ResNet18 and EfficientNet-B0, achieving an accuracy of 90.85%. This indicates that VGG16 performed well at capturing complex features, likely due to its deeper and more sequential structure. However, VGG16's architecture is prone to overfitting, especially given the size of the dataset, which may explain why its performance was more stable but did not increase as quickly as ResNet18's over time. The higher accuracy can be attributed to VGG16's depth, which, although computationally expensive, allows it to learn fine-grained details that improve performance on the task.

4.4 Analysis and discussions

The models used in this study (ResNet18, EfficientNet-B0, and VGG16) each performed reasonably well on the instrument classification task, but their performance varied across different metrics. The training and loss curves, accuracy trends, and confusion matrices provide deeper insight into these results and offer a foundation for understanding the strengths and weaknesses of each model.

The training and loss curves for ResNet18 showed steady progress, with a relatively smooth descent in the loss curve throughout training. This indicates stable convergence, suggesting that the model effectively learned discriminative features from the data.

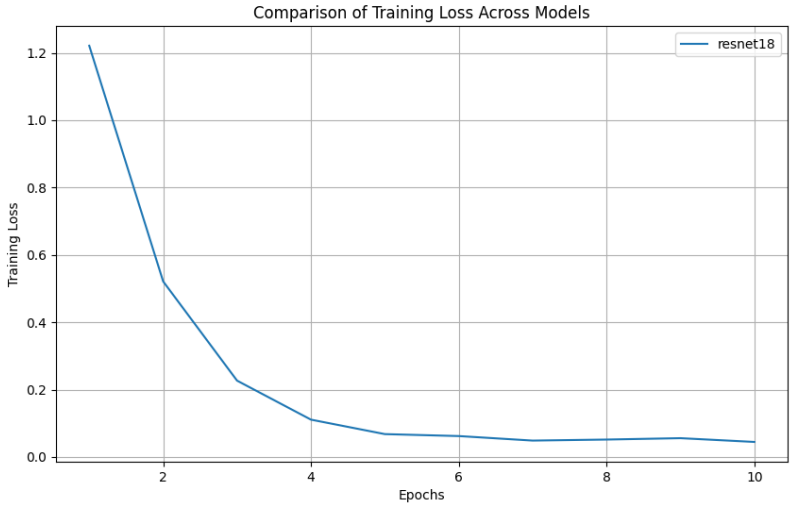


Figure 13: Loss of Resnet18 Model

Despite some slight fluctuations, ResNet18's performance remained robust, likely due to the advantages of its residual connections, which allow the model to avoid common pitfalls like vanishing gradients. The smooth loss curve supports the conclusion that ResNet18 was able to generalize well and avoid overfitting, even in the presence of noise or variability in the dataset. Ultimately, ResNet18 achieved an accuracy of 88.03%, making it one of the stronger models in this study.

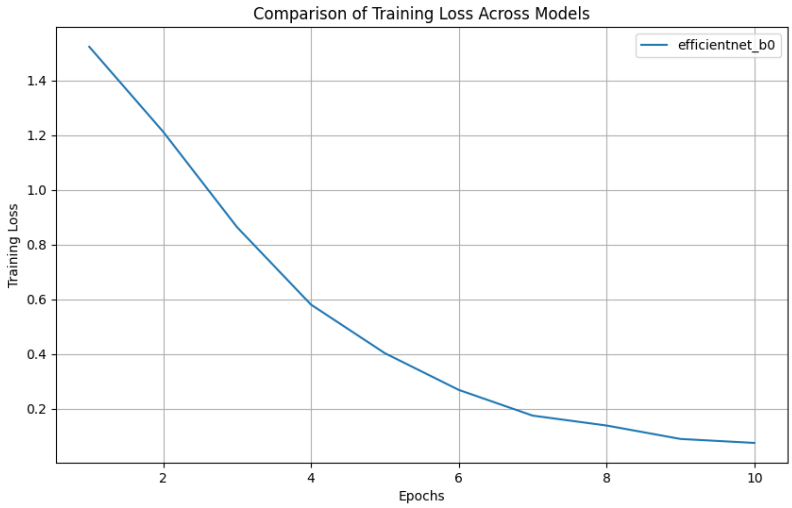


Figure 14: Loss of EfficientNet-B0 Model

EfficientNet-B0 displayed steady progress in its loss curve, although it was slightly less smooth compared to ResNet18. The fluctuations in the loss curve suggest that the model faced challenges in maintaining consistent convergence, which could be attributed to the trade-offs made in the model's architecture for computational efficiency. While EfficientNet-B0 performed efficiently in terms of computational cost, the less smooth loss curve indicates that the model might have struggled to learn optimal features from the data. Despite these challenges, EfficientNet-B0 achieved an accuracy of 86.62%, indicating that, while slightly behind ResNet18, it still demonstrated good performance overall.

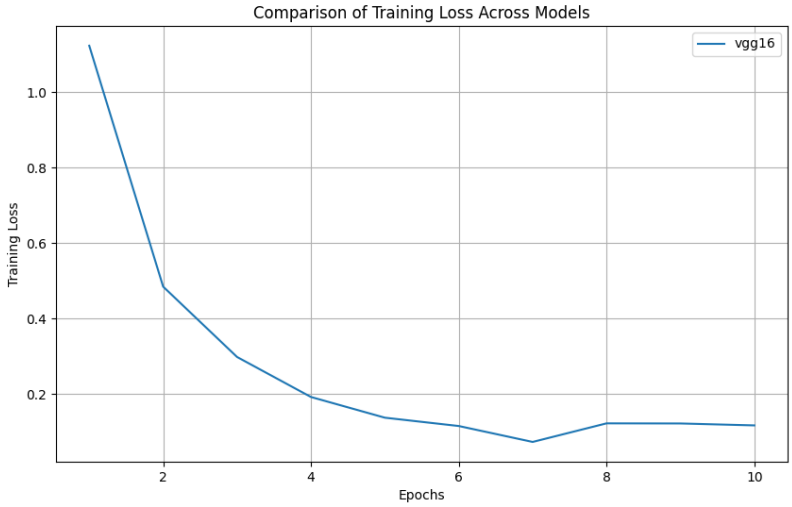


Figure 15: Loss of VGG16 Model

The loss curve for VGG16 was more erratic, with larger fluctuations compared to both ResNet18 and EfficientNet-B0. This instability suggests that the model may have struggled to converge smoothly during training, a common issue for deeper architectures like VGG16. Additionally, VGG16's deeper architecture, which contains a large number of parameters, increases the risk of overfitting, especially with a relatively small dataset like ours. Despite these challenges, VGG16 achieved the highest accuracy of 90.85%, suggesting that its ability to learn complex features outweighed its training instability.

The accuracy curves for all three models reveal that VGG16 consistently achieved higher accuracy compared to EfficientNet-B0 and VGG16. Notably, while EfficientNet-B0 demonstrated rapid improvement early on, its accuracy plateaued before reaching the performance levels of ResNet18. ResNet18's pretrained weights were originally fine-tuned on ImageNet, which may not have been optimal for this audio classification task. Although transfer learning generally provides good results, the specific features learned for ImageNet may not have been perfectly aligned with the spectrograms of musical instruments.



Figure 16: Comparison of Accuracy Across All Models

ResNet18 showed steady loss reduction, indicating that it performed well on the task. The model's residual connections likely contributed to its ability to maintain efficient gradient flow, enabling stable training and good convergence. Despite its relatively lower accuracy compared to VGG16, ResNet18 demonstrated a consistent learning curve, suggesting it effectively learned discriminative features from the data. Its architecture appears to be well-suited for this task, striking a good balance between depth and feature extraction.

EfficientNet-B0 displayed slower progress in terms of accuracy and had a slightly higher loss compared to ResNet18. Although the model performed well, these results suggest that its architecture, designed for computational efficiency, might have limited its capacity to fully capture the intricate patterns in the data. This efficiency-focused design, while beneficial for resource constraints, may have impacted the model's ability to learn effectively for this specific task.

VGG16 showed more instability in its training curves, with noticeable fluctuations in both loss and accuracy. This instability indicates that the model might have struggled to converge smoothly, potentially due to its deeper architecture. While VGG16's depth allows it to learn complex features, it may have led to overfitting, especially with a relatively smaller dataset. Despite this, VGG16 outperformed the other models, suggesting that its capacity for learning fine-grained features outweighed the challenges of instability during training.

The confusion matrices provide valuable insights into where each model is making errors. In the confusion matrix for ResNet18, the model's misclassifications were predominantly concentrated within classes that share similar visual features, such as between different types of string instruments or percussion instruments. This is consistent with the expectation that spectrogram-based features might not be sufficiently distinct for these categories, leading to confusion. While ResNet18 generally provided stable training, its performance was less accurate compared to the other models, likely due to difficulties in distinguishing subtle differences between similar instrument types. It appears that ResNet18 struggled more with distinguishing instruments that share overlapping frequency ranges.

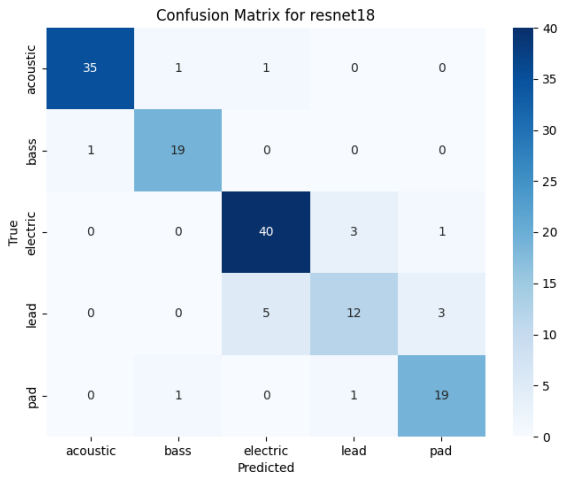


Figure 17: Confusion Matrix for ResNet18

EfficientNet-B0 showed a more balanced performance but its confusion matrix indicated increased false positives and negatives, particularly for instruments with overlapping frequency bands. Despite its computational efficiency, EfficientNet-B0 had a tendency to misclassify instruments that were difficult to differentiate, especially those with subtle differences in their spectrogram features. This suggests that while EfficientNet-B0 is optimized for efficiency, its architecture may have limited its ability to learn fine-grained features, resulting in slightly lower overall performance.

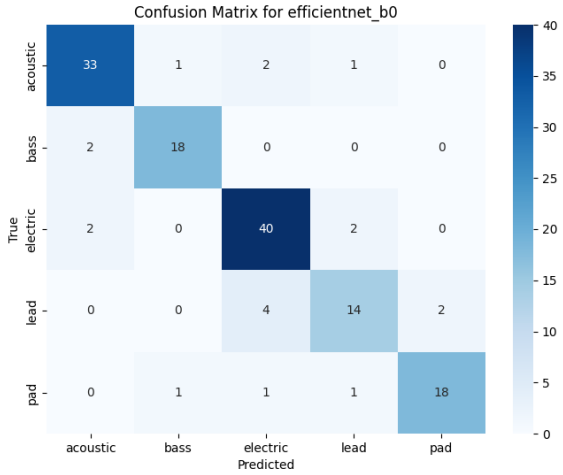


Figure 18: Confusion Matrix for EfficientNetB0

VGG16 showed the least amount of misclassification in its confusion matrix compared to the other models. Its deeper architecture enabled it to capture more detailed and complex features from the spectrograms, leading to more accurate predictions across most classes. Despite the risk of overfitting associated with deeper models, VGG16 performed consistently well without significant instability in training. The confusion matrix revealed that VGG16 was more accurate at distinguishing between the instrument classes, particularly those with subtle differences, which contributed to its superior overall performance.

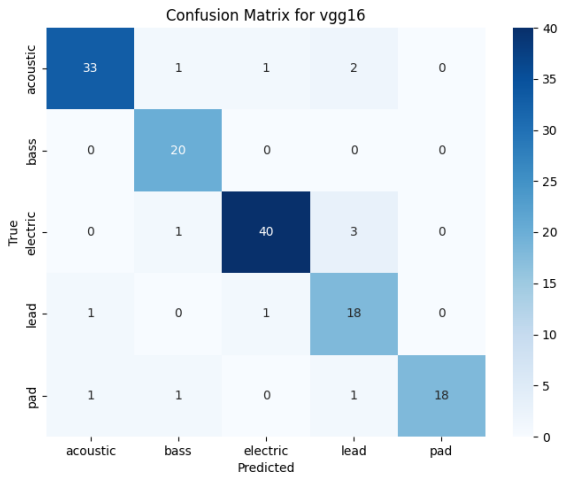


Figure 19: Confusion Matrix for VGG16

The change in performance across different models is an important aspect of this study. While VGG16 performed the best, its computational demands are higher than those of EfficientNet-B0 and ResNet18, which were more efficient in terms of resource usage. EfficientNet-B0's optimized architecture allowed for faster inference, but this efficiency came at the cost of accuracy. ResNet18's architecture, while effective in avoiding gradient issues, may not have been as well-suited for this particular task, where capturing fine-grained spectrogram features was critical.

The models used in this study each demonstrated strong performance on the instrument classification task, but their results varied across different metrics, with VGG16 emerging as the best-performing model.

VGG16 achieved the highest accuracy of 90.85%, with strong precision and recall across most instrument categories. It performed exceptionally well in identifying bass (100.00% recall) and electric (90.91% recall), which contributed to its superior overall performance. Despite its excellent performance, VGG16 had slightly lower precision for lead (75.00%) and a small dip in recall for pad (85.71%), likely due to the risks of overfitting associated with its deeper architecture. This depth enabled VGG16 to capture intricate features in the spectrograms, making it particularly effective at distinguishing between instruments with subtle differences, ultimately making it the top performer in this study.

Classification Report for vgg16:

	precision	recall	f1-score	support
acoustic	0.9429	0.8919	0.9167	37
bass	0.8696	1.0000	0.9302	20
electric	0.9524	0.9091	0.9302	44
lead	0.7500	0.9000	0.8182	20
pad	1.0000	0.8571	0.9231	21
accuracy			0.9085	142
macro avg	0.9030	0.9116	0.9037	142
weighted avg	0.9168	0.9085	0.9099	142

Figure 20: Classification Report for VGG16

In comparison, ResNet18 achieved an accuracy of 88.03% and showed steady progress in terms

of both training loss reduction and accuracy improvement. However, its confusion matrix indicated that it struggled more with distinguishing between instruments with similar frequency ranges, such as lead and pad. The model's residual connections helped stabilize training, allowing it to generalize well, but it was less effective than VGG16 at learning the complex patterns in the spectrograms. The lower performance of ResNet18 can likely be attributed to its difficulty in capturing finer distinctions between certain instrument categories.

Classification Report for resnet18:

	precision	recall	f1-score	support
acoustic	0.9722	0.9459	0.9589	37
bass	0.9048	0.9500	0.9268	20
electric	0.8696	0.9091	0.8889	44
lead	0.7500	0.6000	0.6667	20
pad	0.8261	0.9048	0.8636	21
accuracy			0.8803	142
macro avg	0.8645	0.8620	0.8610	142
weighted avg	0.8780	0.8803	0.8774	142

Figure 21: Classification Report for ResNet18

EfficientNetB0 performed the second best, with an accuracy of 86.62%, but its performance was slightly hindered by its architecture, which is optimized for computational efficiency rather than feature extraction. Although EfficientNetB0 showed steady progress, its loss curve was less smooth than ResNet18's, and its accuracy improvement plateaued earlier. The confusion matrix revealed that EfficientNetB0 had a tendency to misclassify instruments with overlapping frequency ranges, particularly lead and pad, which suggests that its efficiency-focused design limited its ability to capture the complex spectrogram features needed for this task.

Classification Report for efficientnet_b0:

	precision	recall	f1-score	support
acoustic	0.8919	0.8919	0.8919	37
bass	0.9000	0.9000	0.9000	20
electric	0.8511	0.9091	0.8791	44
lead	0.7778	0.7000	0.7368	20
pad	0.9000	0.8571	0.8780	21
accuracy			0.8662	142
macro avg	0.8641	0.8516	0.8572	142
weighted avg	0.8655	0.8662	0.8652	142

Figure 22: Classification Report for EfficientNetB0

Despite the overall positive performance, there were several failure cases, especially in misclassifications between instruments that have visually similar spectrogram patterns. For instance, some stringed instruments were confused with certain percussive sounds, and misclassifications often occurred between instruments that shared similar frequency bands but differed in dynamics. These failure cases highlight the limitations of using spectrograms alone to distinguish between fine-grained audio characteristics.

Another issue was observed with VGG16, where the model struggled with higher variability in the data. VGG16's erratic accuracy and loss curves can be attributed to its deep architecture with a large number of parameters, which increases the potential for overfitting and instability during training. The model's depth allowed it to learn complex features but also made the optimization process more challenging.

5 Conclusion

This project explored the application of convolutional neural networks (CNNs) to audio classification tasks, with a particular focus on the use of spectrograms as input data. The conversion of raw audio signals into spectrograms provided a visual representation that enabled CNNs to analyze frequency patterns and classify different musical instruments based on their unique frequency ranges. After conducting research on the most suitable models for spectrogram analysis, ResNet18, EfficientNetB0, and VGG16 were identified as effective candidates. Transfer learning was applied to these models, which were subsequently trained and tested on a custom dataset of spectrograms. The results indicated that VGG16 outperformed the other models, achieving the highest accuracy, though the differences in performance were minimal.

As the leader of the data pre-processing and transformation, there is one main takeaway. The takeaway is the verification of the ability to extract meaningful data from audio through visual transformations. The use of Mel-spectrograms in use of classifying audio was a success and is a testament to the feasibility of such a practice.

One large area of future consideration is the use of data augmentations to increase model performance in following experiments. Augmentations in the audio domain are useful to increase the robustness of the dataset. Relevant augmentations may include reverbs of various lengths and attributes, distortion, downsampling, audio drop-outs, and more.

Overall, this project provided valuable insights into the process of applying deep learning models to audio classification and the use of spectrograms in feature extraction. The findings contribute to the understanding of model selection and performance evaluation in machine learning applications for real-world audio tasks. Future work sans data representation may focus on optimizing model performance through further hyperparameter tuning, data augmentation, or exploring alternative architectures to improve classification accuracy, particularly in underrepresented categories.

6 Contribution

6.1 Code

I authored all the utility scripts to convert audio into Mel-spectrograms and prepare the data for training (e.g. convertToMelSpectrogram.py and split_data.py) as well as the framework code for training the models (part of train_classifier.py).

6.2 Report

I authored the abstract, introduction, background, method, and the dataset section. My overall contribution was largely in data pre-processing and transformations.

References

[Anw19] Aqeel Anwar. Difference between alexnet, vggnet, resnet, and inception, 2019. Accessed: December 5, 2024.

[Gee24a] Geeks for Geeks. Efficientnet architecture, 2024. Accessed: December 5, 2024.

[Gee24b] Geeks for Geeks. Vgg-16 — cnn model, 2024. Accessed: December 5, 2024.

[HZRS15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.

[SZ15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015.

[TL19] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*, 2019.